Text Manipulation

Information Science
University of Groningen

Leon F.A. Wetzel
l.f.a.wetzel@student.rug.nl

Version 1.0
20-1-2017

1. LINUX

Linux	Operating system based on UNIX. It is widely used by universities, for software development and in supercomputers. The most widely known distribution is Ubuntu.
Operating system	Layer of software that lies between: • Other software and the user; • The computer hardware.
UNIX	Consists of a large number of small, non-interactive programs that perform very specific tasks. Through command line they can be combined into larger programs.

First version of Linux was developed in 1969 in the United States at Bell Labs.

2. STORAGE

At RUG, all Linux machines are connected to a central data storage system.

Data is stored in two types of structures:

• Files	Tekst, audio, image, video, data, etc.
• Directories / Folders	Files and subdirectories.

Every folder is itself part of another folder, except for the **root folder** (/).

Files can be referenced by the folders that lead to it plus the filename itself:

/users/Leon/hoi.txt

3. COMMAND LINE

Common commands

Command line allows you to communicatie with the system by means of commands.

+ Allows you to be agile.
+ Command line is extensible and complementary.
+ Automation and reproducibility (shell script files).
+ Allows you to run jobs on big clusters of computers.
- It takes time and effort to get acquainted.

pwd	‘print working directory’. Shows the name of the current folder.
cd	‘change directory’. Move to another folder.
ls	‘list directory contents’. Shows the names of the files and folders that are in the current folder.

```
$ pwd
/Users/leon
$ cd /Users/leon/software
$ ls
program1.txt program2.txt
```

$	Prompt.
cd	Command.
/Users/leon/software	Command argument
cd /Users/leon/software	Absolute path. A reference to a directory that starts with the root.
cd software	Relative path. A reference to a directory that does not start with the root.
..	Most relative path which refers to the parent of the current directory.
.	Current directory.
ls	Displays a list of files in the current directory.
ls /Users/leon/software	Displays a list of files in /Users/leon/software.
ls -l	Long listing. Shows extra information about the files in the directory in a list view.
ls -a	Displays hidden files and directories as well. Hidden files and directories whose name start with a dot are not displayed.
-l	Option. Options start with a dash and can alter the behaviour of the command.

File permissions

```
$ ls -l software
-rw-r--r--1 leon students 0 Aug 20 14:33 program1.txt
-rw-r--r--1 leon students 0 Aug 20 14:39 program2.txt
```

The following elements can be seen:

1. Permissions;
2. Owner (leon) and group (students)
3. Size in bytes (0)
4. Date and time when the file was last modified
5. File name

Read	E.g. using ls to see which files are in the directory
Write	E.g. add and remove files from the directory.
Run	Go to the directory with cd.

```
-rw-r--r--
```

1. The first character indicates whether the file is a directory (d) or not (-).
2. The next 3 characters show whether the owner can read (r), write (w), run (x) or not (-).
3. The next 3 characters show whether the group can read (r), write (w), run (x) or not (-).
4. The next 3 characters show whether the other users can read (r), write (w), run (x) or not (-).

```
-rw-r--r—
-rw-rw-rw-
-r-xr-xr-x
```

1. File that everyone can read, but only the owner can write.
2. File that everyone can read and write
    a. This could be dangerous!
3. File that everyone can read and run, but no one can write.

```
drwx------
```

A directory in which only the owner can read, write and run.

```
-rw-r--r-
drwxr-xr-x
```

Standard permissions for files and directories.

Permissions can be changed by using the command chmod.

```
$ chmod u+r
$ chmod g-w
$ chmod o+x
```

1. Grants reading permission to the owner.
2. Removes writing permission from the group.
3. Grants running permission to others.

```
drwx–Sr–
```

S indicates that the directory's setgrid is set.

SetGID	When another user creates a file or directory under such a setgid directory, the new le or directory will have its group set as the group of the directory's owner, instead of the group of the user who creates it.
'S'	The directory's setgid bit is set, but the execute bit isn't set.
's'	The directory's setgid bit is set, and the execute bit is set.

Copying files

```
$ cp original.txt copy.txt
```

copy.txt may not exist prior to running cp, in which case it will be created as a copy of original.txt. If it existed previously, its contents will be overwritten.

It is not possible to rename files in Linux, but you can `move' a file to another file name, using the command mv.

```
$ mv original.txt moved.txt
```

Deleting files

```
$ rm program1.txt
```

Files can be removed with the command rm.

Once a file has been deleted, it cannot be recoverd!

```
$ rm -i program1.txt
```

rm -i is a safer way of removing files, because it asks for confirmation before deletion.

Manual

```
$ man ls
```

An overview of the various options that can be used with a Linux command can be found in its manual, which you can read with the command man.

Useful keys when reading manual pages:

- space bar or f (forward) — go to the next page
- b (back) — go to the previous page
- q (quit) — leave the manual (back to the command-line)

4. PROCESSES

Processes	Multiple programs that run simultaneously on Linux.

```
$ ps
PID TTY TIME CMD
5136 pts/0 00:00:00 bash
5160 pts/0 00:00:00 ps
```

ps	Displays a list of processes that are running.
bash	Shell that processes the commands.
ps -fe	Displays additional information and processes.
kill -9 5137	Kills process number 5137.

5. ACCESS TO OTHER MACHINES

ssh	Allows a user to login to another machine. It can also be used to access the home directory remotely at a Linux workplace (LWP).

```
$ ssh s1234567@karora.let.rug.nl
S1234567@karora.let.rug.nl password:
```

yourmachine$ ssh s1234567@karora.let.rug.nl needs a public key!

1. Connect to Bastion

```
yourmachine$ ssh -p 2222 s1234567@bastion.service.rug.nl
```

2. Once logged in to Bastion you can connect to Karora without a public key

```
bastion$ ssh s1234567@karora.let.rug.nl
```

6. HELPFUL COMMANDS

date	Gives the date, day of the week, time zone and time
clear	Clears the terminal window
^C (ctrl-c)	Stops the program that is currently running

Coying and pasting text works in the command-line when opened from a GUI, like this:

1. Place the mouse at the beginning of the piece of text
2. Press the left button
3. Move the mouse to the end of the piece of text keeping the left button pressed
4. Release the left button
5. Move the mouse to wherever you want to copy the text
6. Press the middle button of the mouse

head	Displays the first 10 lines of a file.

tail	Displays the last 10 lines of a file.
tail -n +X	Shows all the lines of a file from line X.
head -n	Displays the first n lines of a file.

```
$ ls -al | head -3
total 25420
drwxr-xr-x 52 leon leon 4096 2010-09-13 10:37 .
drwxr-xr-x 519 root root 20480 2010-09-13 15:12 ..
```

The output of a command can be passed to another command using a pipe (|). This combination of 2 commands show the first 3 lines from the output of ls.

```
$ ls -al | tail
```

See the last 10 lines of the output of ls.

```
$ ls – al | head -3  | tail -1
```

See the third line of the output of ls.

7. COUNTING TEXT ELEMENTS

wc	Word count. Counts the amount of lines, words and characters in a file.
wc -l	Counts all the lines in a file.
wc -w	Counts all the words in a file.
wc -c	Counts all the characters in a file.

wc counts one character per bytecode (8 bits). A character corresponds to one letter, number or punctuation mark for which a separate code is reserved in a computer, this includes also spaces, for example

Words	A word is a string of characters where none of the characters is a whitespace and it is delimited by whitespace characters.
Line	A line is a string of characters between new line characters. The line itself does not contain new lines.

- wc does not count lines but new line characters.

8. SEARCHING IN TEXT

```
$ grep --color vuur file.txt
turfvuurtje op de haardplaat hing een groote ketel water te
vloermat bij het vuur zat een kat met gevouwen voorpooten,
Johannes werd bij het vuur gezet, om zijn voeten te drogen.
```

grep	Can be used to search for tekst strings in text files.
grep --color	Emphasise the strings found in the results.
grep -e word1 -e word2	Search for different words.
grep -i	Case insensitive.
grep -n	Display line numbers in the results.
grep -v	Display only lines without matching (oppositive of default behaviour).
grep -w	Search for words, ignoring sub-word matchings.

```
$ grep -w vuur file.txt | wc -l
```

Displays the amount of lines where the word *fire* occurs.

```
$ grep -w self *.txt
```

Displays the lines of all files in which the word *self* appears.

9. USING THE SHELL

```
$ echo Hello!
Hello!
```

Displays tekst in the screen.

```
$ echo variable PS1 has value: $PS1
Variable PS1 has value: \h \w:
```

Displays the values of shell variables.

The shell where you type your commands is influenced by the values of shell variables. One such variable is PS1, the prompt variable:

```
$ PS1="enter a command: "
enter a command: rm test.pdf
enter a command: PS1="$ "
$ ls
```

Prompts can contain dynamic elements such as the current time, the number of the command or the name of the computer.

```
$ echo $PATH
/bin:/usr/bin
```

The shell variable $PATH contains a list of directories. These are the directories where the shell searches for commands. This shell looks for commands in the directories */bin* and */usr/bin*.

```
$ pwd
/home/s1234567
$ cat .bashrc
# my shell variables:
export PS1='$ ' # $ is not interpreted between ' and '
export PATH=/bin:/usr/bin:/usr/local/bin
```

.bashrc in the home directory ($HOME) contains the values of the shell variables that are frequently used. .bashrc can be edited with a text editor.

```
$ A=1
$ export B=2
$ bash
$ echo $A
$ echo $B
2
```

Exporting definitions of variables causes subprocesses to get them as well. Only $B is known by the subshell.

```
$ cat .bashrc
alias c="clear"
alias mv="mv -i"
alias rm="rm -i"
```

alias — Gives alternative names to commands that are redefined. mv has been redefined as mv -i.

```
$ pwd
/Users/leon/tmp
$ cd .
$ pwd
/Users/leon/tmp
$ cd ..
$ pwd
/Users/leon
$ cd ../..
$ pwd
/
```

.	Current directory.
..	Parent directory.

```
$ ls -al > ls.txt
```

Saves the output of ls -al to a file named ls.txt

- If ls.txt did not exist, it will be created.
- If it did exist, it will be overwritten!

```
$ ls -al > /dev/null
```

Output can be redirected to the special file /dev/null if you want to get rid of the output, for example if you do not want it on your screen or in a file.

```
$ ls -al >> ls.txt
```

Output can also be saved using >>.

>	Overwrites the file.
>>	Appends to the file.

STDERR	Error messages of programs are written to STDERR. STDERR can be redirected by using > and >> preceded by a 2.

```
$ ls -al > ls.out 2> ls.err
```

This command will prevent all error messages from ls to appear in the screen.

STDOUT	Output redirected to ls.out.
STDERR	Errors redirected to ls.err.
‘’	Variables between these quotes are not replaced by their definition.
“”	Variables between these quotes are replaced by their definition.
``	Inverted quotes. The content of the variable will be the output of executing the program.

Quotes always come in pairs!

\’	Quote that will not be interpreted.

```
$ gedit &
```

When starting a process, you can specify that it should run independently of the shell by placing an ampersand (&) after the call.

```
$ ps -ef | grep gedit
```

Check whether a background process is running.

```
$ help cd
cd: cd [-L|-P] [dir]
     Change the current directory to DIR. The variable ...
```

Basic commands such as cd and echo are built into the shell and have no manual.

^D (ctrl-d) Command that stops a shell session.

exit Command that stops a shell session.

10. GREP AND REGULAR EXPRESSIONS

Lines without a certain word with option -v

```
$ grep -v 'word1' file.txt
```

Lines with word1 or word2: with option -e

```
$ grep -e 'word1' -e 'word2' file.txt
```

Lines with both word1 and word2: with a pipe

```
$ grep 'word1' file.txt | grep 'word2'
```

```
$ grep --color -w -e 'dank' -e 'denk' 1949.txt
denk ik aan hen, die vielen als slachtoffer van hun plichtsv...
tekort, dat vooralsnog slechts overbrugd kan worden dank zij...
```

Can be replaced with:

```
$ grep --color -w ‘d[ae]nl’ 1949.txt
```

[ae]	Regular expression. A sequence of characters that define a search pattern. [ae] refers to the characters a and e. The order in this expression does not matter.
[^xyz]	**All characters** except x, y and z.
[bcdfghjklmnpqrstvwxyz]	All consonants.
d[a-z]	Sequences of 2 letters where the first is a *d* and the second any lowercase letter.
[a-z]f	Sequences of 2 letters where the first is any lowercase letter and the second is an f.
[aeiou] [aeiou] [aeiou]	Sequences of 3 lowercase vowels.
[aeiou] [^aeiou] [^aeiou]	Sequences of a lowercase vowel followed by two characters that are not lowercase vowels.

[:digit]	General character group which is the same as [0-9].
[:alpha]	General character group which is the same as [a-zA-Z].
.	**Any character** in a regular expression.
d.k	Describes sequences of 3 characters: the first is *d*, the second is any character and the third character is *k*.
..r.	Describes sequences of 4 characters where the third character is an *r* and the remaing 3 can be any character.
*	Quintifier. Indicates how often a character or a string can occur.
b*	0 or more b's.
ha*	a *h* followed by 0 or more a's.
(ha)*	0 or more times *ha*.
?	0 or 1 time the preceeding character or string.
dank?	Matches the words *dan* and *dank*.
+	1 or more times the preceeding character or string.
e+n	Matches both the words *en* and *een*.
^	Reference to the beginning of a line.
^De	A line that starts with *De*.
!	Reference to the ending of a line.
!$	A line that ends with an exclamation mark.

In regular expressions it is possible to refer back to a previous part of the expression.

```
$ grep '\)[aieou]\)1\1' hoi.txt
```

Searches for the same vowel occurring 3 consecutive times.

```
$ grep '\(.\)\1' hoi.txt
```

Searches for any character that appears twice in a row.

Grep works with lines, but sometimes it is useful to have each word on a separate line.

```
$ grep -E -o ‘\w+’ hoi.txt
```

Searches for strings made of letters and shows the results per word.

How many lines contain the word `de'?

```
$ grep -w 'de' file.txt | wc -l
```

How many times does the word 'de' appear in the text?

```
$ grep -wo 'de' file.txt | wc -l
```

How many words in the text contain an 'a'?

```
$ egrep -o 'nw+' file.txt | grep 'a' | wc -l
```

How many words in the text do not contain any vowel?

```
egrep -o 'nw+' file.txt | grep -v '[aeiouAEIOU]' | wc -l
```

How many times does the letter 'e' appear in the text?

```
grep -o 'e' file.txt | wc -l
```

How many times does the number '1' appear in all texts?

```
cat *.txt | grep -o 1 | wc -l
```

11. REPLACING TEXT

sed

sed	Stream editor. Programming language that can be used to edit files. sed is the name of the command that can be called to edit files from the terminal. Like grep, sed goes through the file line by line!

```
$ sed 's/pattern1/pattern2/options' file1 > file2
```

This command searches in file1 for patern1, replaces it with pattern2 and writes the result to file2.

s Stands for *substitute*.

```
$ sed ‘s/a/A/’ hoi.txt
```

Replaces the first a of each line with an A.

```
$ sed ‘s/[aeiou]//’ hoi.txt
```

Replaces the first lowercase vowel of a line with nothing, actually removing them from the file.

```
$ sed ‘s/cat/dog’ hoi.txt
```

Replaces the first occurrence of the string *cat* in each line with *dog*.

```
$ sed ‘s/a/A/g’ hoi.txt
```

Replaces **all** occurences of *a* by *A* in each line. Lines are corrected from left to right.

```
$ sed ‘s/a/A/i’ hoi.txt
```

Replaces the first occurrence of the letter *a* with an *A*, no matter if it is an uppercase or lowercase letter *a*. This is an example of case insensitive matching.

```
$ sed ‘s/a/&&&/g’ hoi.txt
```

Each a will be replaced with 3 a’s.

The second pattern allows to refer back to the whole first pattern with an ampersand (&).

```
$ sed ‘s/\(.\)\(.\)/\2\1/’ hoi.txt
```

Swaps the first 2 characters on each line.

```
$ grep -E -o ‘\w+’ hoi.txt
```

All the words are put on a separate line, but punctuation is lost! sed can be used to place words and punctuation on separate lines. But how do we do it?

1. Place spaces between punctuation symbols and words.

```
$ sed ‘s/[.!?;,”]/ & /g’ hoi.txt
```

2. Replace each space with a new line.

```
$ sed ‘s/ /\n/g’ hoi.txt
```

Most files will be divided into separate words and punctuation. However, words with an apostrophe and large numbers could cause problems! A context-sensitive tokeniser could solve this problem.

```
$ sed ‘’ hoi.txt
```

Displays all lines of the file.

```
$ sed -n ‘’ hoi.txt
```

Shows nothing.

-n Suppresses automatic printing.

```
$ sed -n ‘1,10p’ hoi.txt
```

Shows only lines from 1 to 10.

```
$ sed ‘/ie/p’ hoi.txt
```

Shows only lines that contain the string *ie*.

tr

tr Translate. Command that replaces characters and groups of characters with other characters or group of characters. tr only reads input from STDIN or through a pipe construction with cat command!

```
$ cat hoi.txt | tr ‘a’ ‘A’
```

Receiving input from the cat command and passing that to tr.

```
$ tr ‘a’ ‘A’ < hoi.txt
```

Replaces each *a* in the file with an *A*.

```
$ tr -d ‘[aeiou]’ < hoi.txt
```

Deletes all lowercase vowels.

-d Option to delete.

```
$ tr ‘cat’ ‘dog’ < hoi.txt
```

In the file, each *c* will be replaces with a *d*, each *a* with an *o* and each *t* with a *g*.

sed	**tr**
Substitutes strings	Substitutes individual characters
Allows you to create context-sensitive patterns	Patterns cannot be context-sensitive
Difficult to replace new line characters	Easy replacement of new line characters
Files can be passed as input argument	Only reads from STDIN through pipe or <

```
$ tr ‘a-z’ ‘A-Z’ < hoi.txt
```

Replaces lowercase characters with corresponding uppercase characters.

```
$ tr -c ‘a’ ‘b’ < hoi.txt
```

Replaces not-a with a b.

-c Complement.

```
$ tr -s ‘[aieou]’ ‘A’ < hoi.txt
```

Replaces all series of lowercase vowels with one A.

-s All series.

```
$ tr ‘\n\t’ ‘ ‘ < hoi.txt
```

Replaces each new line and each tab with a space.

How do we put sentences on separate lines?

1. Place the whole text in 1 line

```
$ tr ‘\n’ ‘ ‘ < hoi.txt
```

2. Place spaces between punctuation characters and words

```
$ sed ‘s/[.!?;,”]/ & /g
```

3. Replace the space after ".", "!" and "?" with a new line

```
$ sed ‘s/[.!?] /&\n/g’
```

```
$ tr ‘\n’ ‘ ‘ < hoi.txt | sed ‘s/[.!?;,”]/ & /g | sed ‘s/[.!?]
/&\n/g’
```

cut

cut Command that can be used to select parts (or fields) of a line.

- cut can be seen as the selection of columns in a table, where grep selects the rows.
- Fields can be separated by different tokens, such as TAB, space, # and many more.
- cut can select specific fields, such as the 9th or the nth.

cut -d # Uses # as field separator.

cut -f1 Selects the first field.

cut -d2-4 Selects the characters mentioned in the second field till the fourth field.

cut -d2,7 Selects the characters mentioned in the second and the seventh field.

```
$ cut -d' ' -f1 hoi.txt
```

Selects the first word of each line.

```
$ cut -d' ' -f2- hoi.txt
```

Selects all but the first word of each line.

```
$ cut -c3,4 hoi.txt
```

Selects the third and fourth character of each line.

uniq

uniq	Command that removes **consecutive** duplicate lines from a text file.
uniq -c	uniq command which also keeps track of how many **successive** duplicates it finds.

```
$ uniq -c < hoi.txt
15 de
7 het
10 van
...
```

uniq only finds duplicates in consecutive lines!

sort

sort	Command which sorts lines according to some order, e.g. alphabetic order.

The order of characters is defined in a table in the shell variable $LANG. The order is:

1.	Spaces	Space
2.	Punctuation	,?!;.
3.	Numbers	1,2,3,4
4.	Uppercase letters	A,B,C
5.	Lowercase letters	a,b,c
6.	Accented letters	á,ë,ò

sort -f	Makes no distinction between uppercase and lowercase letters.
sort -n	Sort according to numerical order.
sort -r	Sort in inverse order, for example B before A.
sort -u	Removes duplicates from the result.

Sort results are written to STDOUT.

paste

paste	Command used for pasting lines of two or more files one afer the other. Results are written to STDOUT.
-d	Customiser for the field separator. The standard field separator is TAB.

```
$ paste -d ' ' caB caB
ccc ccc
aaa aaa
BBB BBB
```

rev

rev	Command which reverses lines at character level.

```
$ echo dit is een test | rev
tset nee si tid
```

rev is useful in combination with cut -c and cut -f.

```
$ cat hoi.txt | rev | cut -c1
```

Selects the last character of each line.

```
$ cat hoi.txt | rev | cut -f1 | rev
```

Selects the last word of each line.

```
$ grep -Eoh ‘\w+’ hoi.txt | sort | uniq | wc -l
```

Returns the amount of different words in a file.

```
$ grep -Eoh ‘\w+’ hoi.txt | rev | sort | rev
```

Returns words, sorted according to their last letters.

12. FREQUENCY LISTS

Frequency lists are useful to carry out analyses of texts. It can be used to automatically determine the language of a document.

```
$ grep -Eo ‘\w+’ hoi.txt | sort | uniq -c | sort -nr
```

Creates a frequency list of words.

```
$ grep -o ‘\w+’ hoi.txt | sort | uniq -c | sort -nr
```

Creates a frequency list of letters.

```
NL words   NL letters      EN words   EN letters
4362 de    65343 e         379 the    4424 e
3096 van   38001 n         249 and    3708 a
1817 en    24380 i         242 of     3091 t
1806 het   23161 a         218 to     3090 i
1329 een   22893 r         216 in     2972 n
```

The frequency of words is useful for determining the language of a document. If you remove function words, the list gives you an overview of the topic of a document.

Tokens "Words" in a text, including duplicates.

Types Unique tokens in a text, without duplicates.

```
$ grep -Eo '\w+' hoi.txt | wc -l
```

Returns the number of tokens.

```
$ grep -Eo 'nw+' j sort -u j wc -l
```

Returns the number of types.

Type-token ratio Number of types divided by the number of tokens.

bc Basic calculator.

The bigger the type-token-ratio, the more diverse the words that are used in a text.

```
$ grep -Eoh 'nw+' /net/corpora/troonrede/*.txt | wc -l > tokens
$ grep -Eoh 'nw+' /net/corpora/troonrede/*.txt | sort -u | wc -l >
types
$ paste -d '/' types tokens | bc -l
.07723904050518191518
```

n-gram Sequence of n elements, for example characters or words.

Uni-grams 1

Bi-grams 2

Tri-grams 3

Uni-grams	Mijn	naam	is	Elsemieke
Bi-grams	Mijn naam	naam is	is Elsemieke	
Tri-grams	Mijn naam is	naam is Elsemieke		
4-grams	Mijn naam is Elsemieke			

How do you make frequency lists of bi-grams?

1. Place each word in a separate line

```
$ sed ‘s/\([^a-zA-Z0-9 ]\)/ & /g’ | tr -s ‘\n\t ‘ ‘\n’ > result1
```

2. Make a copy of the list

```
$ cp result1 result2
```

3. Remove the first word from the copy

```
$ tail -n +2 result2 > result3
```

4. Place the copy next to the first list (horizontally)

```
$ paste -d’ ‘ result1 result3
```

5. Make a frequency list of the result

```
$ echo result3 | sort | uniq -c | sort -nr
```

```
# display all input files
$ cat *.txt |
     # add spaces around punctuation symbols
     $ sed 's/n([^a-zA-Z0-9 ]n)/ & /g' |
     # place each token in a separate line
     $ tr -s 'nnnt ' 'nn' > result1

# display all words starting from the second one
$ tail -n +2 result1 |
# place neighbour words next to each other (the 2nd file comes out
of the pipe)
     $ paste -d' ' result1 - |
# count all word pairs
     $ sort | uniq -c | sort -nr
```

alias	Command for assigning alternative names to a combination of commands.

```
$ alias tokenize="sed ‘s/\([^a-zA-Z0-9 ]\)/ & /g’ | tr -s ‘\n\t ‘
‘\n’”
$ alias freqlist="sort | uniq -c | sort -nr"
```

Aliases can be used instead of a full command.

```
# make word list
$ cat *.txt | tokenize > result1
# make bi-gram list
$ tail -n +2 result1 | paste -d' ' result1 - | freqlist
```

13. SHELL SCRIPTS

General information

Shell script	File which contains a group of commands. It basically is a text file with Linux commands.

```
echo ``Here below you can see the files in this directory''
ls
```

If you run the shell script above, the 2 commands included will be executed sequentially.

chmod	Command for altering user permissions. By altering the user permissions, you can grant running permissions to certain users, allowing shell scripts to run.

```
$ chmod ugo+x lsscript.sh
$ ls -l lsscript.sh
-rwxr-xr-x 1 leon students 72 Oct 9 16:02 lsscript.sh
$ ./lsscript.sh
Here below you can see the files in this directory:
pr0n.tex lsscript.sh mydata.txt
```

You can run shell scripts using /bin/bash as well.

```
$ bash lsscript.sh
Here below you can see the files in this directory:
pr0n.tex lsscript.sh mydata.txt
```

bash lsscript.sh works, because */bin/bash* is included in the variable $PATH. Scripts can also be run using the variable $PATH.

```
$ echo $PATH
/Users/antonio/bin:/opt/local/bin:...
$ cp lsscript.sh /Users/leon/bin
$ lsscript.sh
Here below you can see the files in this directory:
pr0n.tex lsscript.sh mydata.txt
```

Always specify on the first line of a shell script which program should be used to run the shell script!

```
#!/bin/bash
echo ``Here below you can see the files in this directory:''
ls
```

bash is a shell interpreter.

```
#!/bin/bash
# make a list of words and output it to a file
cat *.txt |
     sed 's/\([^a-zA-Z0-9]\)/ & /g' |
     tr -s '\n\t ' '\n' > result1
# make a list of bi-grams using the word list from result1
tail -n +2 result1 |
     paste -d' ' result1 - |
     sort | uniq -c | sort -nr
```

Creates bi-gram lists. However, this is not very efficient by the use of the same temporary file (result1). The script can only process the files *.txt in the current directory as well.

#	Comment line.

It would be great if it possible to call the bi-gram script as follows:

./bigram.sh a.txt b.txt

In shell scripts references can be made to command-line arguments.

$1	First argument	*a.txt*
$2	Second argument	*b.txt*
$*	All arguments	*a.txt* and *b.txt*
$0	The name of the script	bigram.sh

Variable	Temporary storage of data, associated to a name.
TEST	Variable with the name TEST. Names of variables usually consist of uppercase characters.

```
TEST = “abc”
```

Variables can be assigned values by using “=”

```
echo The value of TEST is $TEST
```

The value of a variable can be retrived by preceding the variable name with a dollar sign.

```
RESULT = ‘ls -l’
```

The output of a command can be stored in a variable.

The inverted quotes indicate that the command between them will be run. The output of the command is then placed in the variable.

Two instances of the same script can run concurrently on Linux. Therefore, temporary files should not have fixed names!

mktemp Command which generates a new for a new temporary file.

```
$ mktemp
/mnt/D/tmp/tmp.rHfHQ573pT
FILE =’mktemp’
```

A new file name has been generated. The filename can be placed in a variable, as seen on the last line.

```
#/bin/bash # run as: ./bigram.sh file

# 1. define temporary file
FILE=`mktemp`

# 2. create a word list and save it to a temporary file
cat $* |
sed 's/\([^a-zA-Z0-9]\)/ & /g' |
tr -s '\n\t ' '\n' >
$FILE

# 3. create a bi-gram list based on the word list
tail -n +2 $FILE |
paste -d' ' $FILE - |
sort | uniq -c | sort -nr

# 4. remove temporary file
rm -f $FILE
```

Output of shell scripts can be redirected in the same way as commands, such as using echo, > and >>.

There are 2 types of output that shell scripts can generate:

- Normal output, goes to STDOUT.
- Error messages, go to STDERR.

read A — Input from a keyboard or file can be read and will then be placed in variable A.

```
$ read ANSWER
Welcome! # typed by the user

$ echo You have typed: $ANSWER
You have typed: Welcome!
```

Conditional statements

if — Sometimes commands should be run only if certain conditions are met. This is possible with the command if:

```
read A
if [ $A=1 ]
then
      echo You have typed 1
else
      echo You have not typed 1
fi
```

Different conditions can be used with if.

- Comparisons

```
$A = $B                 # (is equal to)
$A != $B                # (not equal to)
```

- String tests

```
-z $A                   # (empty string)
-n $A                   # (non empty string, i.e. its length is > 0)
```

- File tests

```
-e $FILE                # (file exists)
-f $FILE                # (file is a regular le)
-d $FILE                # (file is directory)
```

Repeating commands

The command *while* is a variant of *if* in which commands are run until the condition is false.

```
while [ -n "$1" ]
do
      echo first argument of the script: $1
      shift # discard the first argument
done
```

This example checks all command-line arguments and shows them on the screen shift discards the first argument. As a consequence, the second argument becomes the first, the third becomes the second and so forth.

Commands can be run multiple times with the command *for*.

```
for FILE in *.txt
do
      NEWNAME=’echo $FILE | sed 's/txt$/xml/'’
      mv $FILE $NEWNAME
done
```

Every file with extension .txt is renamed to have extension .xml instead.

xargs

xargs	Each line of STDIN (standard input) is passed as argument to the given command.

```
$ cat hoi.txt
a
b
c

$ cat hoi.txt | xargs echo
a b c

$ cat hoi.txt | xargs -n 1 echo
a
b
c
```

14. TABULAR FILES

CELEX	A collection of tables containing information about words in 3 different languages.

An example is the file dpw.cd with 6 columns:

1. Element id
2. Word
3. ID of the lemma
4. 3 fonetical representations of the word

Backslashes are used as column delimiters.

```
1\a\1\'a\[a:]\[VV]
2\a\2\'a\[a:]\[VV]
3\Aafje\3\\\
4\Aafke\4\\\
5\Aagje\5\'ax-j@\[a:x][j@]\[VVC][CV]
```

Sample data from the document dpw.cd.

Linux commands can be used to carry out several actions for this document:

• Row selection	grep
• Column selection	cut
• Row selection according to column value	cut, paste and grep
• Combining columns	Not easy!

```
# What is the element with id 23?
grep '^23\\' dpw.cd

# What are the most frequent CV combinations?
cut -d'\' -f6 dpw.cd | sort | uniq -c | sort -nr | head

# List all words that have the lemma with ID 7
grep '[^\\]*\\[^\\]*\\7\\' dpw.cd
or
cut -d'\' -f3 dpw.cd | paste -d'\\' - dpw.cd | grep '^7\\'

# Replace the lemma column by the id of the corresponding word
# (very difficult to solve with Linux commands)
```

15. CONCORDANCES

Concordance program	Creates an index, similar to an index of a book.
KWIC	Keyword in context.
KWIC index	Form of concordance. An index which contains words with their contexts.

```
n een maghrebijns gezin in Marseille degene die het hoofd koel houdt
. @NH1995
0144-3-15|Twee uitdrukkingen waarmee degene die zich ervan bedient ,
wil aangeve
75-4-4|Anderen zullen hem kennen als degene die samen met Piet Vroon
zijn doctor
133-5-1|Voor kinderen die boffen met degene die voor de klas staat ,
zijn de les
18-0005-8-3|Van alle liberalen heeft degene die het meest consequent
oppositie v
een draad snot hangt uit de neus van degene die verdriet heeft om
het afscheid e
```

How do you build a KWIC index?

1. Use grep to find the relevant lines in the text
2. Use sed to highlight the match with special symbols

```
#the character `#' cannot not appear in the text
$ grep pattern | sed -e 's/\(pattern\)/#&#/' > hits
```

3. Use cut to pick the match as well as its left and right contexts

```
$ cut -d# -f1 hits > lc
$ cut -d# -f2 hits > match
$ cut -d# -f3 hits > rc
```

4. Use sed to make the left and right contexts long enough
    a. Two TABS were used here!

```
$ sed -e 's/^/        /' < lc > lc2
$ sed -e 's/$/        /' < rc > rc2
```

5. Use cut -c to make the left and right contexts short enough

```
$ cut -c1-30 < rc2 > rc3
$ rev lc2 | cut -c1-30 | rev > lc3
```

6. Use paste to show the match and its left and right contexts

```
$ paste lc3 match rc3
```

16. XML

XML Extensible Markup Language. A collection of markup languages with which internal structure can be added to documents. XML files usually contain text, but they can also contain other types of information such as tables, images, sound, animations, links, etc.

```
<text>
 <head>
  <sentence>
   CDA in gespannen sfeer bijeen
  </sentence>
 </head>
 <start>
  <sentence>
   De CDA-fractie is al urenlang bijeen voor overleg over de
formatie.
  </sentence>
  <sentence>
   De sfeer in de fractie is volgens ingewijden gespannen.
  </sentence>
  ...
 </start>
 <source>NOS Teletext</source>
 <date>31-08-2010</date>
</text>
```

Example of an XML document.

XML documents contain tags. Tags are case sensitive!

<abcd>	Opening tag.
</abcd>	Closing tag.
<abcd/>	Equivalent to <abcd></abcd>
<a>Hoi.	Tag nesting.

The structure of an XML document can be represented as a tree.

Opened tags must always be closed before their parent tags are closed.

Correct: <a>

Incorrect: <a>

In XML, it is NOT allowed to open tags and not close them! All open tags must be closed.

The philosophy of XML is to use descriptive tag names so that a document can be presented in different ways.

Additional information can be added to a tag, using attributes:

```
<sentence number="23" function="main clause">
```

- Attributes follow the name of the tag.
- They consist of a name (here number en function), followed by the sign = and a value delimited by quotes.
- The attributes of a tag must have unique names!

```
<text source="NOS Teletekst" date="31-08-2010">
 <head>CDA in gespannen sfeer bijeen</head>
  <sentence>
   de CDA-fractie is al urenlang bijeen voor overleg over de
formatie .
  </sentence>
  <sentence>
   de sfeer in de fractie is volgens ingewijden gespannen .
  </sentence>
  ...
</text>
```

Better example of an XML document.

XML documents should start with a prologue, which are tags with information about the document.

```
<?xml version=”1.0” encoding=”UTF-8”?>
```

This tag indicates that the file contains an XML document using XML version 1.0, and that the characters are encoded using the character set UTF-8.

<? ?>	Tag construction which is called processing instructions.

All letters, numbers and other characters are represented internally in computers as sequences of bits, i.e. ones and zeros. The conversion from bits to characters is different per character set.

ASCII	95 tokens represented with 7 bits (US English).
ISO 8859-1	191 tokens represented with 8 bits (25 languages).
UTF-8	About 100000 characters represented with a variable number of bits.

Entity	A name in an XML document that references an object. It can be compared with a variable (constant) in programming languages.

XML entities start with an ampersand (&), followed by a name and \;", for example: &test;

All entities except **amp**, **apos**, **gt**, **lt** and **quot** must be defined, for example as follows:

```
<!DOCTYPE rootelement
 [
  <!ENTITY test1 "the text for entity test1">
  <!ENTITY test2 "the text for entity test2">
 ]
>
```

rootelement is the name of the root tag of the document.

In HTML spaces and newlines are skipped. Conversely, XML does take spaces and newlines into account. The content of the element <a> A contains 3 characters!

```
<section>
<!-- This is a comment! -->
 <text>Hoi.</text>
</section>
```

XML documents are used as input in various types of programs: editors, web browsers, databases, ...

All these programs include an XML parser: a software routine that recognises XML tags and can convert XML documents in trees containing tags and their associated information.

XML parsers are very strict: if the XML format of a document is not correct then the processing of that document is aborted.

xmllint Program which checks the syntactic structure of an XML document.

```
$ xmllint hoi.xml
```

+ Standard method to add structure to documents.
+ Documents are easily interchangeable between users.
+ Strict definitions simplify the quality control of documents.
+ Extra structural elements allow to present the document in different ways.

- XML annotations take up a lot space and XML documents are therefore much bigger than the information contained in them.
- Data supplied in a wrong XML structure is unusable.
- Searching in XML documents is typically slow.

17. XSLT

General information

XSLT — Extensible Style Language for Transformation (.xsl). Programming language that is used to convert XML documents to other formats (often HTML). The language consists of instructions that define certain actions for XML elements.

```
<xsl:template match="play">
 <font style="color: blue;">
  <xsl:apply-templates/>
 </font>
</xsl:template>
```

Changing the text colour.

XSLT programs look like XML files. XSLT files can contain a mix of tags, such as XML and HTML and these files cannot be validated.

xsltproc — Command which converts XML files into HTML files by using an XSLT file.

```
$ xsltproc program.xsl input.xml > output.html
```

An XSLT file can also be defined inside an XML file.

```
<?xml version="1.0" encoding="utf8"?>
<?xml-stylesheet type="text/xml" href="program.xsl"?>
...
```

In this case the style file does not need to be passed as an argument to xsltproc:

```
$ xsltproc input.xml > output.html
```

The XML document can now be also used in a web browser.

```
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
 <xsl:template match="/">Hello world!</xsl:template>
</xsl:stylesheet>
```

This program contains 1 instruction for the root (/) of a document. It replaces the XML document with the line Hello world!

Nodes

Node — Elements, attributes, text, comments, namespaces and processing instructions.

/ — Root of a document. The root node stands above all nodes!

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- markup added by: Leon -->
<play>
 <text speaker="LYSANDER">
 'k Weet waarlijk niet waarom 'k zoo somber ben,
  …
```

The root node has two children:	comment and <play>
The node <play> has 1 child:	<text>
The node <text> has 2 children:	attribute speaker and text

XSLT instruction	Edits the element that is referred to in the attribute *match*.

```
<xsl:template match=”description”>…
```

Within the instruction something must be done with the contents of the element, otherwise it will not be present in the output produced by XSLT.

```
<xsl:template match=”description”/>
```

This instruction discards the content inside all elements with the name *description*.

If XLST encounters an XML element for which no instruction has been defined, it performs the default action:

- Child elements are processed with instructions available.
- Text content is made visible.
- All other information is omitted: element names, attributes, comments and processing instructions.

```
<xsl:template match="element">
     <xsl:apply-templates/>
</xsl:template>
```

Processes child elements.

```
<xsl:template match="text()">
     <xsl:value-of select="."/>
</xsl:template>
```

Shows text content.

```
match="element"
```

All elements with name *element*.

```
match="parent/element"
```

All elements with name *element* and parent *parent*.

```
match="ancestor//element"
```

All elements with name *element* and ancestor *ancestor*.

```
match="element[@attribute='value']"
```

Elements *element* whose attribute *attribute* has the value *value*.

```
<xsl:apply-templates/>
```

Process the content of the current element.

```
<xsl:apply-templates select="path"/>
```

Process the content of the element with path *path*.

```
<xsl:value-of select="path"/>
```

Shows the contents of the element with path *path*.

The current element can be referred to with *select="."*

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:output method="html" encoding=''UTF-8''/>

<xsl:template match="/">
 <html>
  <body>
   <p><xsl:apply-templates/></p>
  </body>
 </html>
</xsl:template>

<xsl:template match="description | text">
 <p>
  <xsl:apply-templates/>
 </p>
</xsl:template>

</xsl:stylesheet>
```

Example of a XSLT program.

```
<xsl:template match="text/description">
<xsl:apply-templates/>
</xsl:template>
```

XSLT also allows context sensitive instructiions. This instruction only affects the elements *description* that have as parent an element *text*.

XSLT uses the most specific instructions for each element.

Styling

```
<xsl:template match="description">
 <p style="padding:5px; color:white; background-color: black;">
     <xsl:apply-templates/>
 </p>
</xsl:template>
<xsl:template match="text/description">
     <font style="color:red;"><xsl:apply-templates/></font>
</xsl:template>
```

Style definitions such as colour and font type can be stored in a separate style file or in the elements themselves.

```
<xsl:value-of select="@speaker"/>
```

Text from the XML document can be made visible using the XSL element value-of, for example for the attribute *speaker*. The character @ indicates that we do not refer to the element but to the attribute.

```
<xsl:number count="text"/>
```

XSLT keeps track of the number of elements processed. The instruction displays how many <text> elements have been processed at the current time.

With XSLT it is easy to present HTML structures in the input XML file.

- Place the information in XML elements.
- Define how the elements are to be converted to HTML.

Conditional and loop statements

```
<xsl:template match="text">
 <xsl:if test="@speaker='LYSANDER'">
  ...
 </xsl:if>
</xsl:template>
```

... will only run for text elements whose speaker attribute is LYSANDER.

There is NO else statement in XSLT!

```
<xsl:choose>
 <xsl:when test="@speaker='LYSANDER'">
  ...
 </xsl:when>
 <xsl:otherwise>
  ...
 </xsl:otherwise>
</xsl:choose>
```

This instruction allows for multiple conditions (<xsl:when) and 1 generic alternative (<xsl:otherwise).

```
<xsl:for-each select="text">
     <xsl:value-of select="@speaker"/>,
</xsl:for-each>
```

This instruction displays a list of speakers, separated by commas.

- With <xsl:for-each we can show a list of specific information from an XML document.
- XPath statements are used in order to count from this type of information.

Functions

```
<xsl:template name="menu">
 <p>
     <a href="item1">Item 1</a> |
     <a href="item2">Item 2</a>
 </p>
</xsl:template>
```

Functions are useful to identify pieces of code that are more frequent in the document.

You can refer to the code by using a name, for example:

```
<xsl:call-template name="menu"/>
```

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
 <xsl:output method="text" encoding="UTF-8"/>
 <xsl:template match="sentence">
      <xsl:value-of select="."/>
      <xsl:text>
      </xsl:text>
 </xsl:template>
</xsl:stylesheet>
```

Only returns one sentence.

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
      xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
 <xsl:output method="text" encoding="UTF-8"/>

 <xsl:strip-space elements="alpino_ds node"/>

<xsl:template match="node[@word and @lemma]">
 <xsl:value-of select="@word"/>
 <xsl:text> </xsl:text>
 <xsl:value-of select="@lemma"/>
 <xsl:text>&#xA;</xsl:text>
</xsl:template>

</xsl:stylesheet>
```

Only returns words and lemmas.

18. CREDITS

Based on the course sheets of the course Text Manipulation by Antonio Toral.

This summary was created by Leon Wetzel in academic year 2016 – 2017.